# UmbraCrawl
# Stealth Web Crawling Framework — Architecture Research Document

Source survey: Scrapling, Crawlee, Scrapy, Colly, Camoufox, Patchright, undetected-chromedriver, Cra

2026-10-10

**Scope.** UmbraCrawl design for a new stealthy crawler combining HTTP impersonation, engine-level anti-detect browsers, adaptive parsing, and production orchestration. Monochrome edition: all 21 figures use black lines on white only. Sources are the ten cloned repositories listed in Section 2.

## Contents


**1 Goals and Threat Model** **3**
- 1.1 Goals . . . 3
- 1.2 Threat model . . . 3

**2 Source Survey: What Is Taken From Each Repository** **3**

**3 System Architecture** **4**
- 3.1 Component responsibilities . . . 5

**4 Flow 1: Fetch Escalation** **5**

**5 Flow 2: Stealth Handshake** **6**

**6 Flow 3: Adaptive Parsing** **7**

**7 Flow 4: Session and Proxy Lifecycle** **8**

**8 Flow 5: Deployment View** **8**

**9 Flow 6: Scheduler Queue Pipeline** **9**

**10 Flow 7: Challenge Detection Decision Tree** **10**

**11 Flow 8: Fingerprint Assembly** **10**

**12 Flow 9: Proxy Health Loop** **11**

**13 Flow 10: Browser Pool Manager** **11**

**14 Flow 11: Leak Check Suite** **12**

**15 Flow 12: Pipeline Validation Chain** **12**

**16 Flow 13: Local Run Sequence (CLI)** **13**

**17 Flow 14: Retry Budget and Error Handling** **13**

**18 Flow 15: Selector Relearn Loop** **14**

**19 Flow 16: Data Model** **14**

**20 Flow 17: Observability and Autoscale** **15**

**21 Flow 18: Threat Coverage Map** **15**

**22 Flow 19: Versioned Build Phases** **16**

**23 Flow 20: Local Research Loop (OpenCode)** **16**

**24 Stealth Matrix** **18**

**25 Verification Plan** **18**

**26 Build Order: v0.1 to v1.0** **18**

# 1 Goals and Threat Model

## 1.1 Goals

1. Crawl clear and bot-protected targets with one router: fast HTTP path by default, browser path only on challenge.
2. Keep ban rate low via session isolation, proxy health tracking, and per-profile fingerprint consistency.
3. Survive layout drift via multi-candidate selectors with scoring and auto-relearn snapshots.
4. Emit dual outputs: validated item JSON and LLM-ready Markdown.
5. Operate as a queue of workers with observable metrics: requests/s, challenge rate, leak score, parse recall.

## 1.2 Threat model

Adversaries: Cloudflare, DataDome, Imperva, PerimeterX / Human, Akamai. Signals they use:

1. Network: TLS ClientHello fingerprint (JA3), HTTP/2 SETTINGS order, header order and casing.
2. Runtime: `navigator.webdriver`, CDP command artifacts, missing browser plugins, WebGL renderer strings.
3. Behaviour: instant navigation, no mouse/scroll, fixed viewport, datacenter ASN, cookie reuse across identities.
4. Content: honeypot links, obfuscated class names, JS-required bodies.

Design rule: any identity (session + proxy + fingerprint) is used consistently for its lifetime and retired on challenge failure, never mixed.

# 2 Source Survey: What Is Taken From Each Repository

All repositories were cloned shallow on 2026-10-10 into the working directory. Table 1 summarises reuse.

Table 1: Reuse matrix. Take = port the pattern; Leave = consume as dependency or discard.

| Repo (local dir) | Take | Leave |
|---|---|---|
| D4Vinci-Scrapling `scrapling/fetchers/`, `parser.py` | Fetcher router HTTP → impersonate → browser; adaptive selector scoring; `curl_cffi` + `browserforge` usage. | Spider/CLI wrapper; reimplement scheduler interface. |
| apify/crawlee-python `src/crawlee/` | SessionPool, ProxyConfiguration, RequestQueue dedupe, AutoscalingPool, Router table. | SQLite-backed clients; same interfaces, local files. |
| apify/crawlee JS `packages/browser-pool/` | Browser retire/relaunch policy; per-page injection hooks; header ordering from `got-scraping-client`. | Node transports; port to `asyncio`. |
| scrapy/scrapy `core/`, `middlewares/` | Middleware chain, politeness, `dupefilters.py`, LinkExtractor rules, Settings priorities, feed exports. | Twisted engine; rebuild on `asyncio`. |
| gocolly/colly `colly.go, queue/` | `OnRequest/OnHTML/OnResponse` callbacks; throttling queue; `unmarshal.go` struct mapping. | Go runtime; HTTP-only limits. |
| daijro/camoufox `pythonlib/camoufox/` | Engine-level fingerprint model, locale/font/viewport presets, WebGL/canvas addons, launch args. | Build system (`multiversion.py`); use as dependency. |
| Kaliiiiiiiiii-Vinyzu/patchright `patchright.patch` | CDP leak patch list; driver patch method; `detection_smoke_test.mjs` as regression template. | Fork maintenance; consume patched driver. |
| ultrafunkamsterdam/undetected-chromedriver | Patcher concept reference (`patcher.py`, `cdp.py`) only. | Runtime (stale 2025-07-05); Selenium blocking model. |
| unclecode/crawl4ai | Dispatcher fan-out; Markdown generation, content filter, chunking; `async_configs.py` knobs. | Monolithic crawler class; split into stages. |
| mendableai/firecrawl `apps/api/` | Local run shape (prompt → crawl → cited answer); scraper/service boundary; single-SQLite storage pattern. | Server stack (Redis, Postgres, SDK fleet); local CLI replaces the gateway. |

## 3 System Architecture

Figure 1 shows the layered view. Every request passes through identity binding before any network I/O.

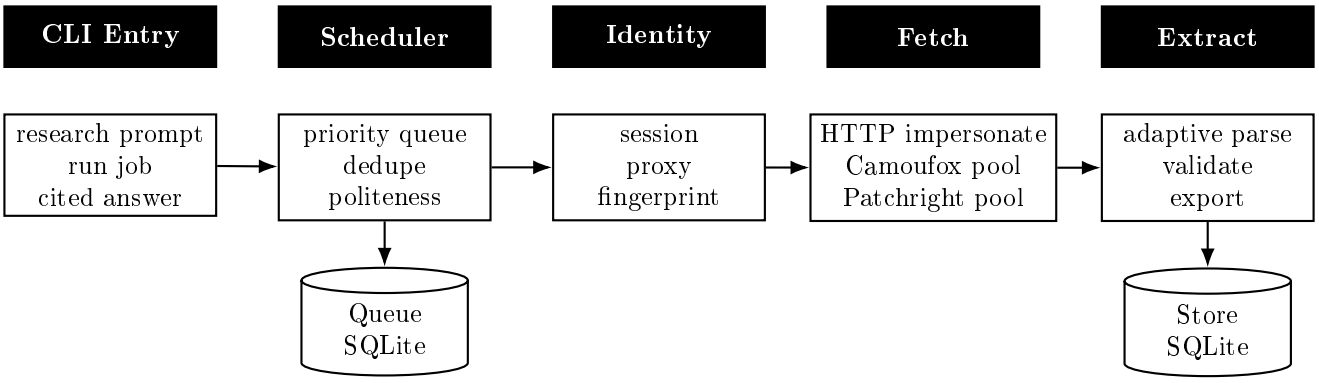



Figure 1: Layered architecture. Identity binding precedes fetch; storage is behind the pipeline, never beside it.

### 3.1 Component responsibilities

Table 2: Module contract summary.

| Module | Input → Output | Failure mode |
|---|---|---|
| `cli/` | Prompt → cited answer rows | Reject empty prompt, no crawl started. |
| `scheduler/` | URLs → leased tasks | Drop duplicates, backoff per-host. |
| `identity/` | Task → bound (session, proxy, profile) | Retire identity on 2 challenges. |
| `fetchers/` | Bound task → raw response | Escalate HTTP → browser, else poison. |
| `parser/` | HTML → items + Markdown | Snapshot + relearn on zero-match. |
| `pipeline/` | Raw items → clean rows | Quarantine on schema fail. |

Proposed file tree:

```
umbracrawl/
  cli/             prompt.py, run.py, answer.py
  scheduler/     queue.py, politeness.py, dedupe.py
  identity/      sessions.py, proxies.py, fingerprints.py, leakcheck.py
  fetchers/      http_impersonate.py, camoufox_pool.py,
                 patchright_pool.py, router.py
  parser/        adaptive.py, selectors.yml, markdown.py
  pipeline/      validate.py, clean.py, export.py
  storage/       sqlite.py (tasks, items, identities; FTS5)
  observability/ metrics.py, traces.py
  tests/         leak_suite.py, challenge_fixtures/
```

## 4 Flow 1: Fetch Escalation

Figure 2 is the core cost saver: the cheap path handles clear targets, the browser path handles only challenges.

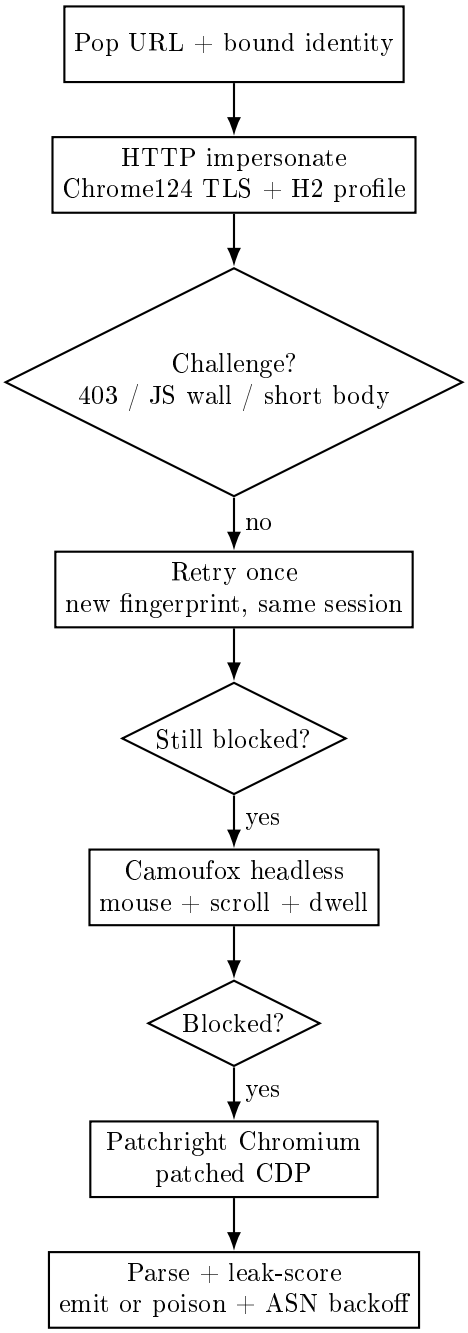



Figure 2: Fetch escalation. Browser stages execute only after HTTP fails twice.

Rules: maximum two HTTP attempts per identity; browser stages reuse the same proxy so behaviour stays consistent; three consecutive browser failures poison the URL and cool down the ASN.

## 5 Flow 2: Stealth Handshake

Figure 3 shows identity binding order. Order matters: fingerprint first, then proxy, then network.

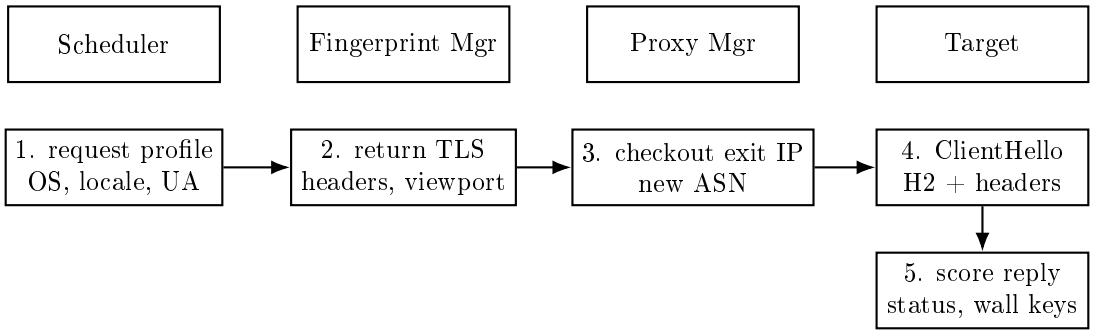



Figure 3: Handshake order. No request leaves without a bound triple (session, proxy, profile).

Consistency checklist per identity: same User-Agent and Sec-CH-UA across redirects; same viewport and timezone; same exit IP for the session lifetime; cookies isolated per session file.

## 6 Flow 3: Adaptive Parsing

Figure 4 covers layout drift, the Scrapling-derived stage.

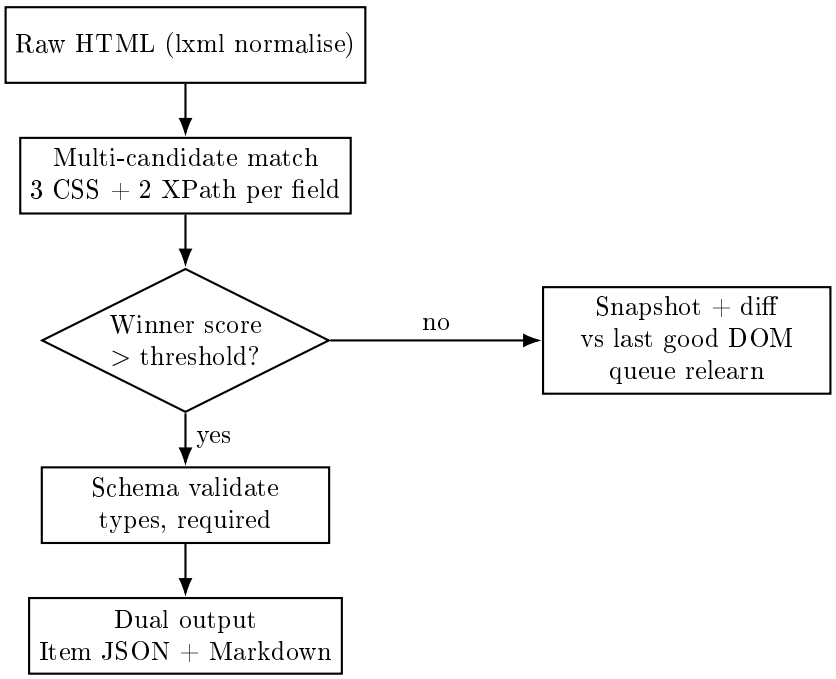



Figure 4: Adaptive parse. Zero-match never drops silently; it snapshots for relearning.

Scoring inputs: candidate text density, DOM depth stability across samples, historical win rate stored in `selectors.yml`. Threshold default 0.65, tuned per spider.

## 7 Flow 4: Session and Proxy Lifecycle

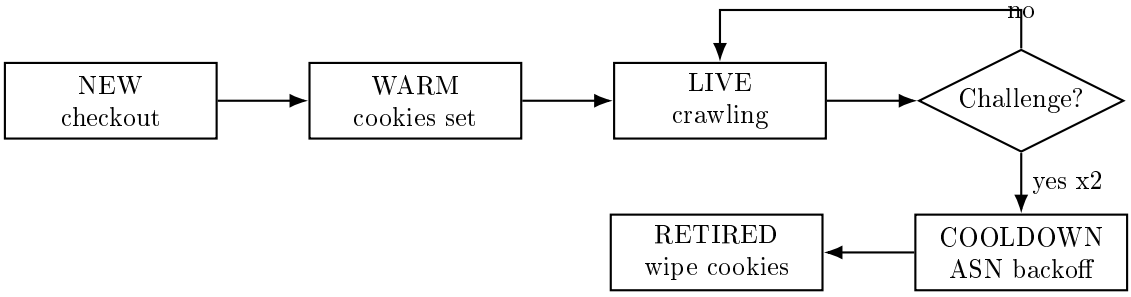



Figure 5: Identity lifecycle. Two challenges retire the identity; cooldown protects the ASN pool.

Proxy policy: residential exit per session; maximum 30 requests per IP per host per hour (tunable in `politeness.py`); health probe every 60 s; failed ASN cooled for 15 min.

## 8 Flow 5: Deployment View

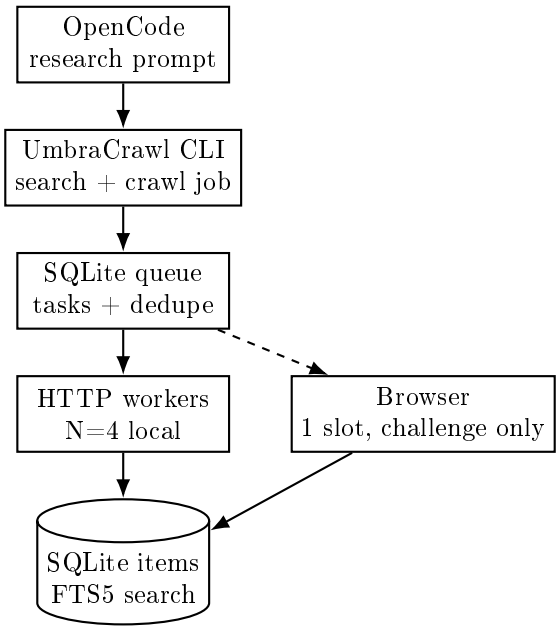



Figure 6: Local deployment. No servers: SQLite holds queue, dedupe, and items; the browser runs only on challenge.

Local-first, single machine: 4 workers max against a SQLite task table (WAL mode). The browser borrows one of the 4 slots on challenge pages, so concurrency never exceeds 4. No Redis, no Postgres, nothing to deploy. The `storage/` interface stays abstract so a server backend can replace SQLite later without touching fetchers or parser.

## 9 Flow 6: Scheduler Queue Pipeline

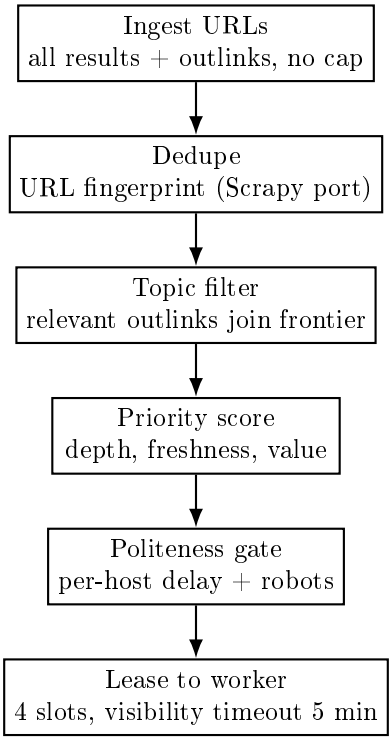



Figure 7: Scheduler stages. Uncapped intake; a URL crawls only after dedupe, topic filter, scoring, and politeness all pass.

## 10 Flow 7: Challenge Detection Decision Tree

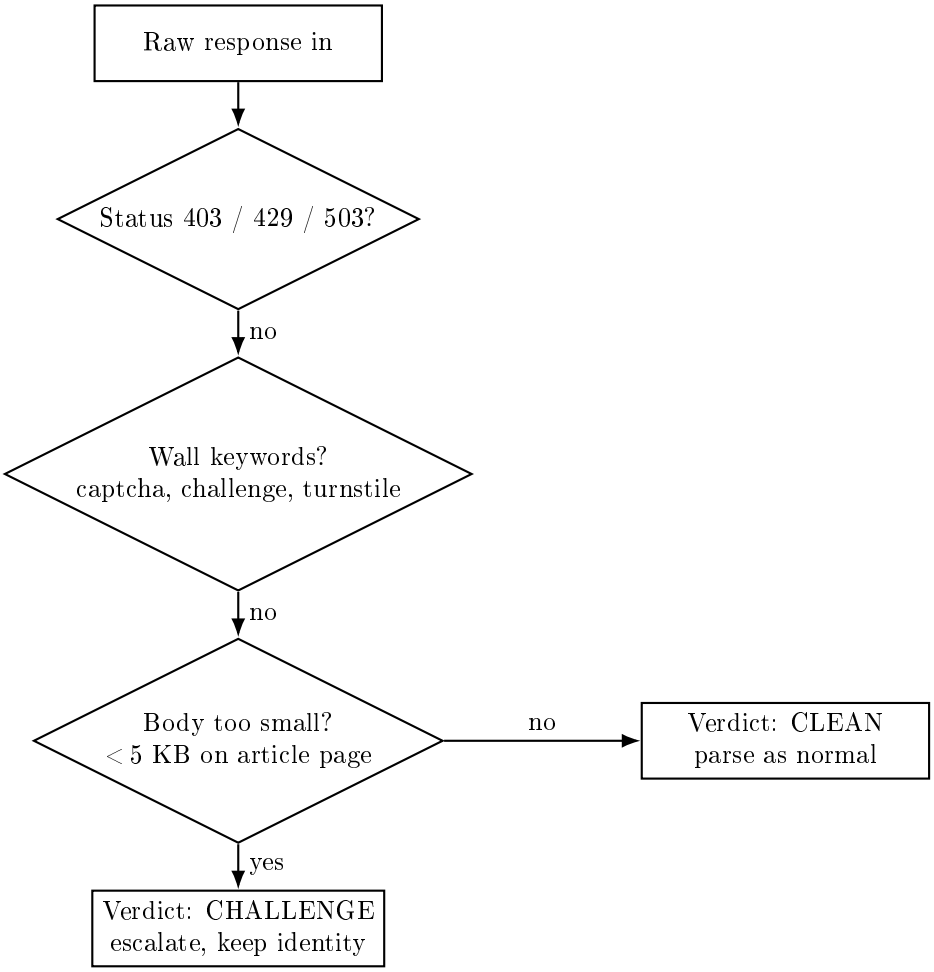



Figure 8: Challenge detector. Any yes on the three tests routes to the browser path.

## 11 Flow 8: Fingerprint Assembly

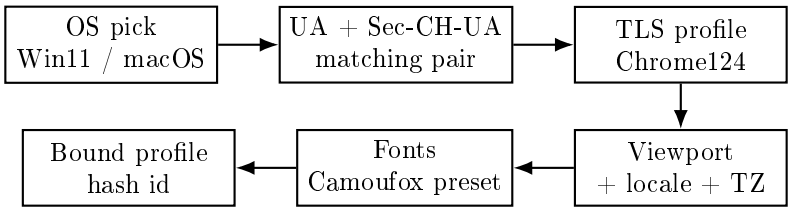



Figure 9: Fingerprint assembly. Every field derives from one OS choice so the profile stays self-consistent.

## 12 Flow 9: Proxy Health Loop

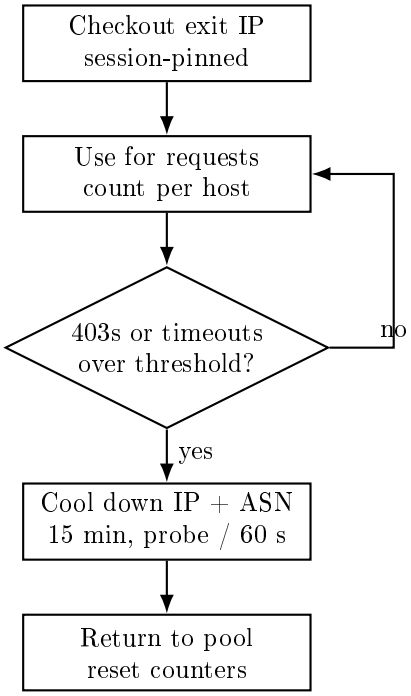



Figure 10: Proxy health loop. Failures cool the IP and its ASN together to stop cluster-wide burns.

## 13 Flow 10: Browser Pool Manager

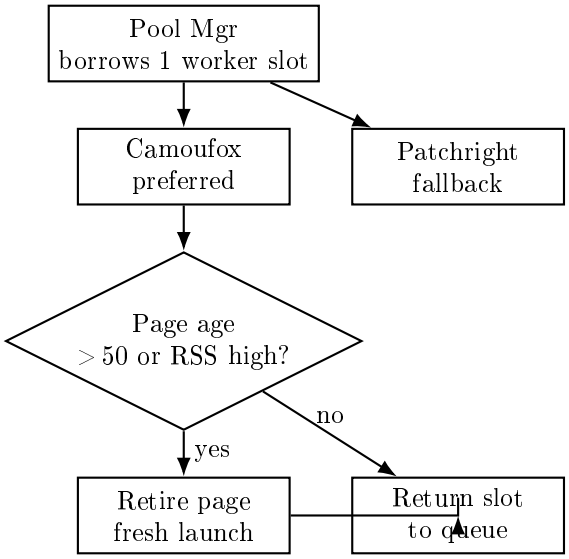



Figure 11: Browser slot. One worker slot converts to a browser page on challenge; the other three keep crawling.

## 14 Flow 11: Leak Check Suite

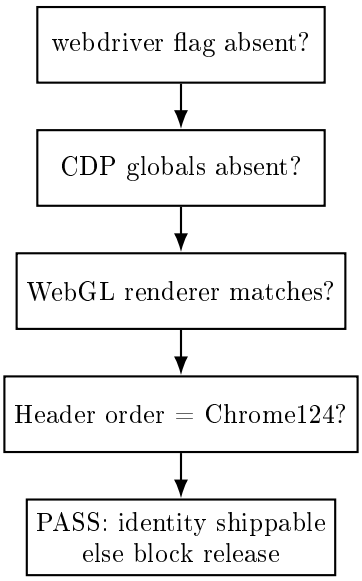



Figure 12: Leak gate. Ported from Patchright smoke test; a new profile ships only after all four pass.

## 15 Flow 12: Pipeline Validation Chain

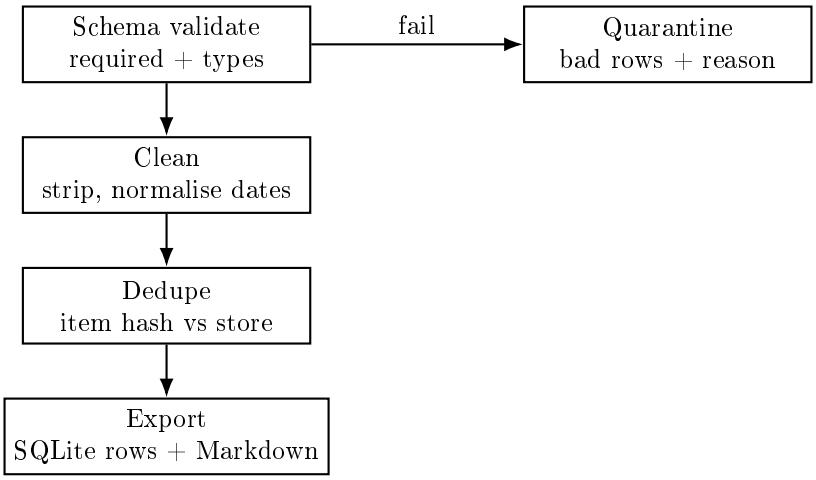



Figure 13: Export pipeline. Failures quarantine with reasons; good rows fan out to both stores.

## 16 Flow 13: Local Run Sequence (CLI)

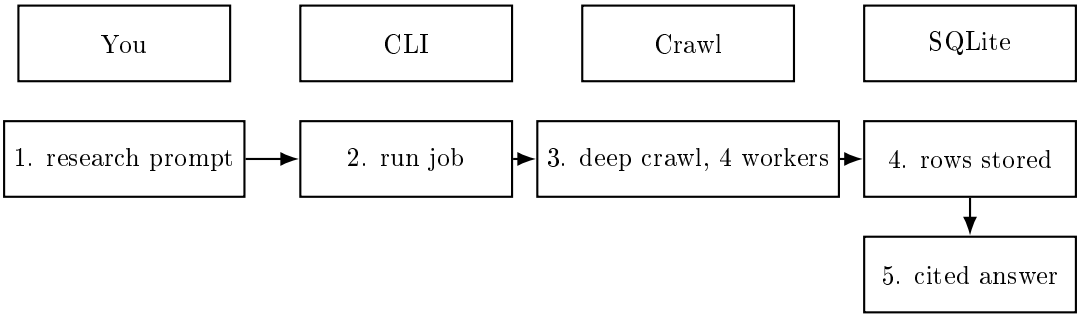



Figure 14: Local run contract: prompt in, deep crawl on 4 workers, cited answer out. No servers, no webhooks.

## 17 Flow 14: Retry Budget and Error Handling

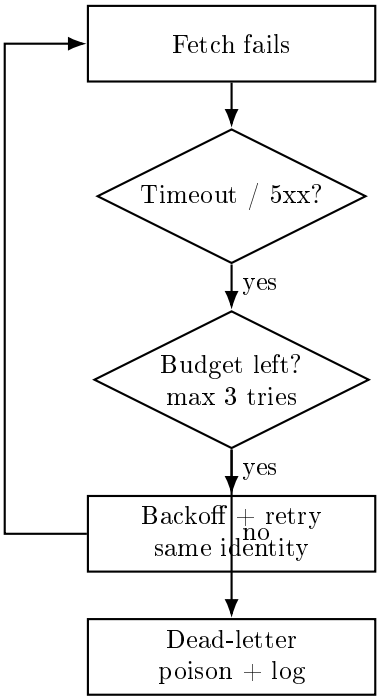



Figure 15: Retry budget. Only transient errors retry; challenges leave this loop for the escalation path.

## 18 Flow 15: Selector Relearn Loop

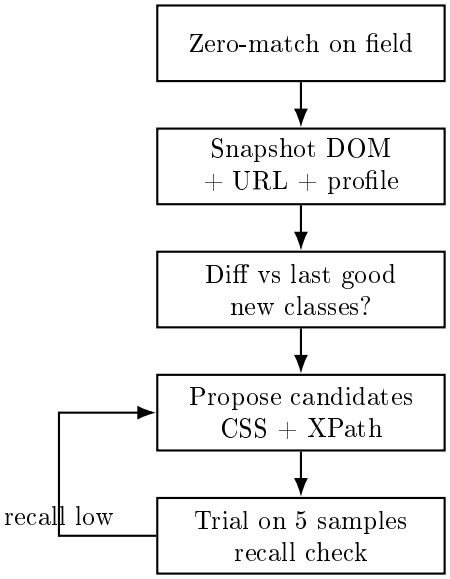



Figure 16: Relearn detail for Figure 4. Candidates graduate to `selectors.yml` only after trials.

## 19 Flow 16: Data Model

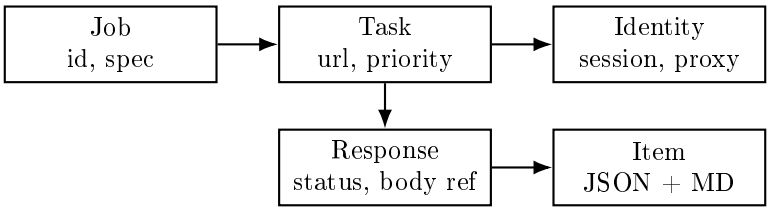



Figure 17: Core records. One job owns many tasks; each task binds one identity and yields one response.

## 20 Flow 17: Observability and Autoscale

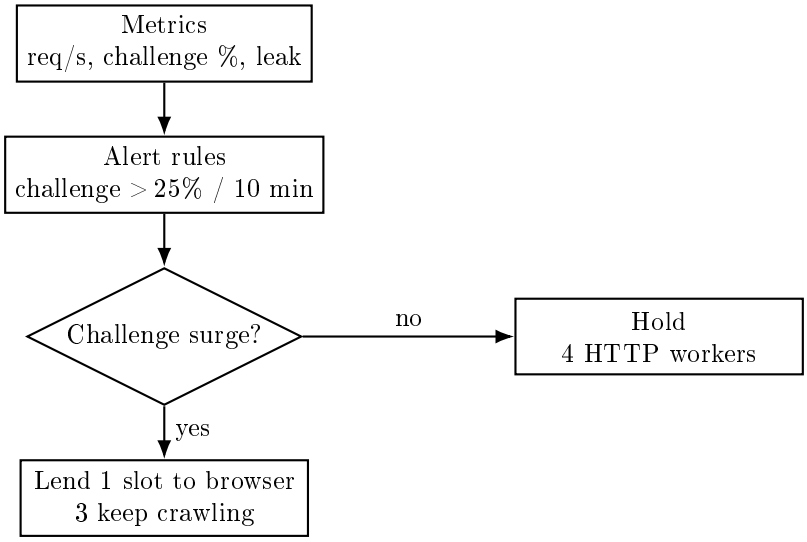



Figure 18: Control loop. Fixed 4-worker budget: challenge surges only move a slot to the browser, never add workers.

## 21 Flow 18: Threat Coverage Map

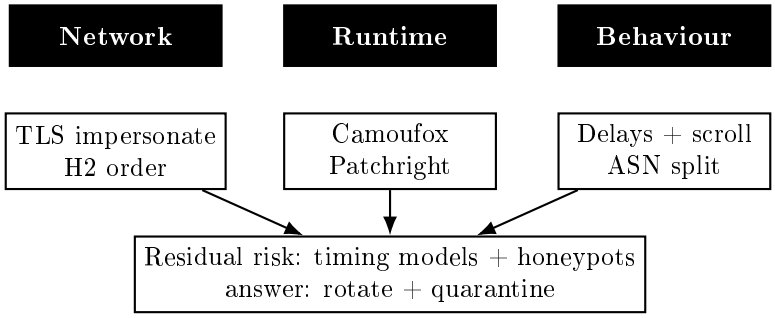



Figure 19: Coverage map. Each adversary signal class has one owning defence; residuals are handled by rotation.

## 22 Flow 19: Versioned Build Phases

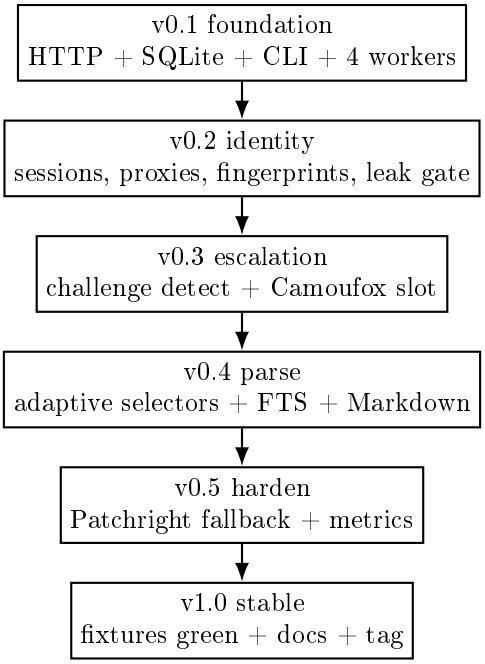



Figure 20: Version ladder. Each version ships a working researcher; nothing lands half-wired.

## 23 Flow 20: Local Research Loop (OpenCode)

Your use case, end to end. You type `research a book` in OpenCode; UmbraCrawl does the legwork and hands structured rows back for the answer. No page cap: it works the full frontier for as long as it takes, on 4 workers.

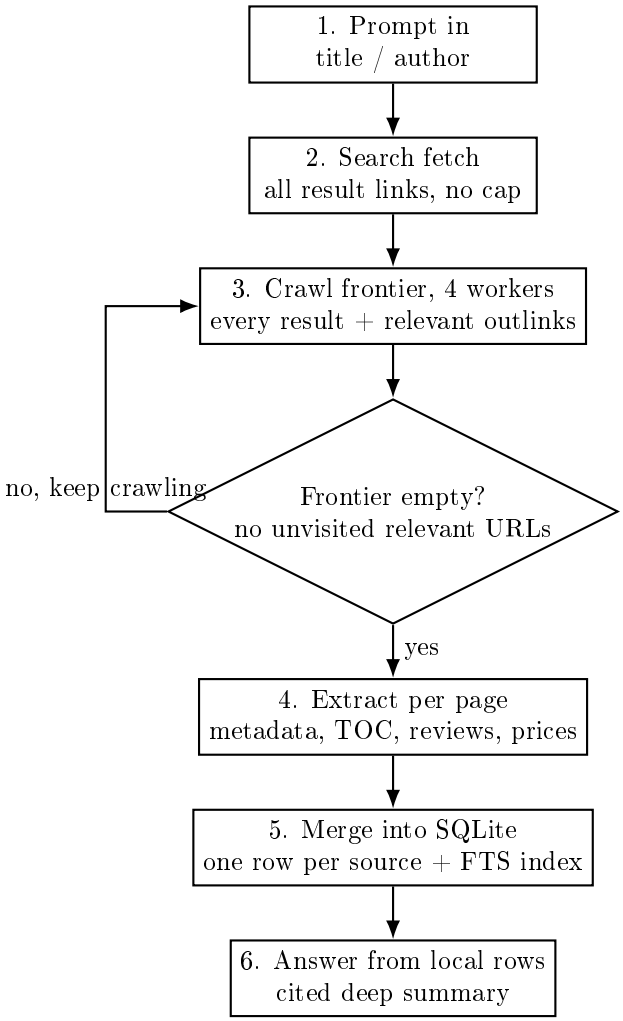



Figure 21: Research loop. Uncapped: the frontier drains fully before extraction merges and the answer composes.

Depth rules: every search result is crawled, and each page's outlinks join the frontier when they match the topic filter (same book, author, publisher, reviews). Only 4 workers run at a time; time is unbounded. Repeat queries hit SQLite first: if the book's sources are already stored and fresh, step 6 answers with zero network. Stale rows (older than the freshness window in `politeness.py`) re-queue only their URLs.

## 24 Stealth Matrix

Table 3: Technique versus signal. B/W print: pass = closed square, partial = half, fail = open (described in text).

| Technique | Defeats | Caught by |
|---|---|---|
| TLS + H2 impersonation (`curl_cffi`) | JA3 / header-order filters | JS challenges, Turnstile. |
| Camoufox engine patches | Canvas, font, WebGL, `webdriver` checks | Behavioural timing, mouse entropy. |
| Patchright CDP patches | CDP command probes (Cloudflare) | Version drift if unpatched. |
| Residential rotation + ASN split | IP rate limits, ASN bans | Cost; still needs politeness. |
| Humanised behaviour (scroll, dwell) | Headless heuristics | Long-run timing models. |

## 25 Verification Plan

1. **Leak suite** (`tests/leak_suite.py`, from Patchright smoke test): assert `navigator.webdriver` undefined, CDP globals absent, WebGL renderer matches profile, headers order matches Chrome124.
2. **Challenge fixtures**: recorded Cloudflare / DataDome pages; assert router escalates and recovers body $> 5$ KB.
3. **Parse recall**: per-field precision/recall on 50 sampled pages; assert recall $\geq 0.95$ after relearn.
4. **Ban metrics**: challenge rate, 403 share, proxy retry share, ASN cooldown events; alert if challenge rate $> 25\%$ over 10 min.
5. **Load test**: 4 workers against local fixtures; record pages/min and p95 fetch latency.

## 26 Build Order: v0.1 to v1.0

1. **v0.1 foundation.** HTTP fetcher (`curl_cffi`), SQLite task/item tables, scheduler with dedupe + politeness, CLI research loop uncapped on 4 workers. Done when a book prompt returns cited rows.
2. **v0.2 identity.** Sessions, proxy support, `browserforge` fingerprints, leak gate (Figure 12). Done when all four leak checks pass.
3. **v0.3 escalation.** Challenge detector (Figure 8) + router + Camoufox borrowed slot (Figure 11). Done when fixtures recover bodies $> 5$ KB.
4. **v0.4 parse.** Adaptive multi-candidate selectors, relearn loop (Figure 16), FTS5 index, Markdown output. Done when recall $\geq 0.95$ on samples.
5. **v0.5 harden.** Patchright fallback, proxy cooldowns, metrics + alerts. Done when challenge rate $< 25\%$ on test set.
6. **v1.0 stable.** Fixture suite green, docs complete, version tag cut. No new features; only fixes.